

Pipeline Hazards

Pipelining lets you hide the latencies of instructions by running them concurrently, but also creates some potential obstacles of its own – characteristically called *pipeline hazards*, that is, situations when the next instruction cannot execute on the following clock cycle.

There are multiple ways this may happen:

- A *structural hazard* happens when two or more instructions need the same part of CPU (e.g., an execution unit).
- A *data hazard* happens when you have to wait for an operand to be computed from some previous step.
- A *control hazard* happens when a CPU can't tell which instructions it needs to execute next.

The only way to resolve a hazard is to have a *pipeline stall*: stop the progress of all previous steps until the cause of congestion is gone. This creates *bubbles* in the pipeline – analogous with air bubbles in fluid pipes – a time-propagating condition when execution units are idling and no useful work is done.



Pipeline stall on the execution stage

Different hazards have different penalties:

- In structural hazards, you have to wait (usually one more cycle) until the execution unit is ready. They are fundamental bottlenecks on performance and can't be avoided – you have to engineer around them.
- In data hazards, you have to wait for the required data to be computed (the latency of the *critical path*). Data hazards are solved by restructuring computations so that the critical path is shorter.
- In control hazards, you generally have to flush the entire pipeline and start over, wasting a whole 15-20 cycles. They are solved by either removing branches completely, or making them predictable so that the CPU can effectively *speculate* on what is going to be executed next.

As they have very different impacts on performance, we are going to go in the reversed order and start with the more grave ones.